

Morning Cheat Sheet — read this and nothing else

Software Engineer · Lloyds · Java, Spring Boot, Microservices · Learn nothing new today — revision only.

45 minutes. Nothing else. No new material.

If a line here does not ring a bell, open that file for three minutes and come back.

The 8 answers that carry this interview

1. **HashMap**: "`hashCode()` , then a spread of `h ^ (h >>> 16)` , then bucket index `(n-1) & hash` . Collisions chain in a linked list, and since Java 8 a bucket over 8 entries becomes a red-black tree. Default capacity 16, load factor 0.75, so it resizes and rehashes at 12."
2. **equals/hashCode**: "Equal objects must have equal hash codes. Override `equals` without `hashCode` and the map looks in the wrong bucket and never finds the entry."
3. **N+1**: "One query for the list, then one per row for the relation. Find it by counting queries in the SQL log. Fix with `JOIN FETCH` , `@EntityGraph` , batch fetching, or a DTO projection."
4. **@Transactional internal call**: "It works through a proxy. An internal `this.method()` call bypasses the proxy, so no transaction starts." Rolls back on **unchecked** exceptions only.
5. **Constructor injection**: "Fields can be final, dependencies cannot be missing, and I can test with `new Service(mock)` without a Spring context."
6. **Microservices trade**: "You buy independent deployment, scaling and fault isolation, and you pay with network failure, no distributed transaction and harder debugging." **Say the cost — that is the part that scores.**
7. **Mocking**: "I mock what I do not own or what makes the test slow — vendor clients, repositories. Not the class under test."
8. **Index cost**: "Faster reads, but every insert and update maintains every index, so I add them deliberately."

• The four honesty answers — rehearse these most

Microservices depth → "On Tradu I built and consumed REST services inside a Spring Boot backend and worked with external vendor APIs. I have not owned a multi-service architecture end to end." **Then immediately**: the trade-off in answer 6 above. Name your level first — it buys credibility for everything else.

Kafka / AWS / Docker / Jenkins → "Exposure rather than depth. I know the Kafka model — topics, partitions, consumer groups, why you publish an event instead of calling synchronously — but I have not run it in production." **Do not get flattered into overclaiming.**

Role ended July 2026 → Two sentences, the real reason, what you have been doing since.
No apology. Two months is unremarkable; evasiveness is not.

The AI tools line → "I use them to think out loud with, but I do not commit anything I could not explain and defend myself. And in a regulated environment you cannot paste proprietary code or customer data into a public tool — I would follow the organisation's policy on approved tooling." **The data point is the one that matters.**

Core Java

- **Overloading** = compile time, different parameters.
Overriding = runtime, subclass.
You cannot override a static method — it is hidden.
- **Abstract class** = shared state and code, one only.
Interface = capability contract, many. *Payment vendors = interface. Base transaction = abstract class.*
- **ArrayList** O(1) get · **LinkedList** O(n) get. Default to ArrayList.
- **HashMap** not thread safe · **Hashtable** locks the whole map, legacy · **ConcurrentHashMap** locks per bucket.
- **String immutable** — string pool, security, thread safety, cached hash.
- `==` references, `.equals()` values. `new String("a") != "a"` .
- **Checked** = compiler forces handling · **unchecked** extends `RuntimeException` .
- **finally** always runs except `System.exit()` . Never `return` from `finally` .

Java 8

- **Intermediate** ops lazy (`filter` , `map` , `sorted`) · **terminal** trigger (`collect` , `reduce` , `findFirst`).
- `map` one-to-one · `flatMap` one-to-many then flattened.
- `Predicate` → filter · `Function` → map · `Consumer` → forEach · `Supplier` → lazy.
- `groupingBy(X, counting())` · `groupingBy(X, summingDouble(Y))` .
- **Optional**: `map` / `orElse` / `orElseThrow` , not `isPresent` + `get` .

Spring Boot

- **IoC** = framework controls creation · **DI** = how it hands dependencies in.
- Beans are **singleton by default** → keep them **stateless**.
A mutable field is a concurrency bug and, on a financial platform, an audit failure.
- `@Repository` also **translates database exceptions**.
- `@RestControllerAdvice` = one error shape. Log the detail, return a generic message.
- **Auto-configuration** = `@ConditionalOnClass` + `@ConditionalOnMissingBean` . Your bean wins.
- `@WebMvcTest` slice · `@SpringBootTest` full · `@MockBean` replaces a bean.

JPA and Hibernate

- **JPA** spec · **Hibernate** implementation · **Spring Data JPA** the layer on top.
- `@ManyToOne` / `@OneToOne` default **EAGER** — set them **LAZY**.
`@OneToMany` / `@ManyToMany` **LAZY**.
- **Dirty checking** — a managed entity needs no `save()`.
- `@Enumerated(EnumType.STRING)`, never **ORDINAL**. Money is **BigDecimal**, never `double`.
- **REQUIRES_NEW** = the audit row survives when the main transaction rolls back.

Microservices

- **Discovery** so addresses are not hard-coded · **API gateway** = one entry point, auth and routing in one place · **database per service** = no joins, no distributed transaction.
- **Circuit breaker**: closed → open → half-open. Failing fast protects both sides.
A payment vendor that hangs must not take your service down.
- **Always a timeout**. Only retry **idempotent** operations — never a blind payment retry.
- **Saga** = local transactions + **compensating** transactions. Eventual consistency.
- Split by **business capability**, not technical layer.
Strangler fig, never big-bang.

Security

- **JWT** = header.payload.signature. **Signed, not encrypted** — nothing sensitive in it.
- Stateless, so any instance validates it — that is why it suits microservices.
- Authorisation **server side**. A UI check only hides the button.
- **BCrypt** — slow and salted by design.
- Parameterised queries, not concatenated SQL. Secrets out of the repo.

SQL

- **Left-most prefix**: an index on `(status, created_at)` does not serve `created_at` alone.
- Index killers: a **function on the column**, a **leading %** in **LIKE**.
- **EXPLAIN**: `type: ALL` = full scan. Check `key`, `rows`, `Extra`.
- **WHERE** before grouping · **HAVING** after.
- Deep **OFFSET** is slow → **keyset pagination**.

Testing and quality • (your strong ground)

- **Unit** = isolated, mocked, fast · **integration** = wired together, proves the pieces fit.

- **Mock** what you do not own or what is slow. `@Mock` + `@InjectMocks` ·
`when().thenReturn()` sets up · `verify()` checks a side effect · `ArgumentCaptor` asserts on what was passed.
- Test **behaviour and rules**, especially the **negative** cases — duplicate request does not debit twice, below-minimum is rejected.
- **Coverage** measures what ran, not what was verified.
- **Code review order**: correctness → error handling → data layer → tests → security → readability last.
- **Logging**: at the boundaries, with a traceable id, correct levels, **never sensitive data**.
- **SonarQube**: catches the mechanical issues so review can spend its time on design. Quality gate on new code.

Your stories

Tradu (FXCM) — trading platform: onboarding, identity verification, transaction processing across multiple payment vendors. Java/Spring Boot backend, React frontend, AWS.
What you did: backend business components, controllers and services · REST APIs consumed by React · Spring Security, validation, exception handling · JUnit 5 and Mockito · refactoring · improved logging · defect verification and root cause analysis · Jenkins, SonarQube, Postman, Swagger.
The vendor story is your best one — external systems you do not control, so timeouts, idempotency and not debiting someone twice.

Opening and closing

"Tell me about yourself" → Now → What you built → **Why this role**. Under 2 minutes.

"Why Lloyds?" → Domain fit — nearly two years on a trading platform doing onboarding, verification and payments. Plus:
"a product built and run long term changes how you write things — you care about tests, logging and maintainability differently when you will be supporting it in two years."

Your closing question → *"What is the biggest technical problem the team is working on right now?"*

The last five minutes

- **You are immediately available**. Most candidates are serving 60-90 days. Say it.
- **Name your level before they ask**. It makes everything else you say credible.
- **If you do not know something: say so, then reason out loud**. That has never cost anyone an offer. Bluffing has.
- Give the **why**, not just the **what**.
- Sit up. Breathe out slowly. Smile before you speak — it changes your voice.

Go get it.